Du BIM au SSI neuro-symbolique — rapport

Exercice CIFRE TwinHypervisor (Université Gustave Eiffel) · Emerick Lafortune · 12 novembre 2026 Dépôt : `elafortune/twin_hypervisor` · reproduction : `python -m twinhyp run --ifc data/duplex_TwinHypervisor.ifc --params data/parametres.json --obs data/observations.csv --out sorties`

1. Architecture de la chaîne

Six modules Python indépendants (`src/twinhyp/m1_…` à `m6_…`), qui ne communiquent **que par les fichiers de sortie** ; chacun se relance seul (`--etapes`). Chaque sortie est validée contre son contrat (JSON Schema du kit, formes SHACL publiques F1–F8 et 11 formes « maison ») à la fin de l'étape qui la produit : une sortie non conforme arrête la chaîne. Aucune valeur n'est codée en dur : tout vient de `parametres.json` (les clés ajoutées ont un défaut dans le code et sont recopiées dans le nœud `ssi:Parameters` du graphe).

Module	Entrée → sortie	Utilité	Méthode
M1 jumeau géométrique	IFC → `geometry.json`	décrire le bâtiment de façon mesurable et vérifiée : espaces, portes, sorties, liaisons, distances, voxels, positions de pose, anomalies	IfcOpenShell ; connectivité par vote de 3 sources ; graphe de visibilité + Dijkstra ; lancer de rayons vers les plafonds
M2 graphe	`geometry.json` → `ontology.ttl`, `kg.ttl`	représenter le bâtiment et ses règles de sécurité dans une ontologie interopérable, interrogeable (SPARQL) et vérifiable (SHACL)	rdflib ; socle `ssi:` aligné sur BOT, SOSA, SAREF4BLDG
M3 IWCMC	voxels → `pareto.json`	choisir où poser les nœuds IoT et mesurer ce qu'apportent les matériaux réels du jumeau	modèle multi-murs, PREP-NSGA, 4 scénarios
M4 déploiement SSI	`geometry.json` → `devices.json`, `kg.ttl`	placer le minimum d'équipements qui respecte les règles de détection, d'alarme et de vidéo	MILP exacts (HiGHS)
M5 évacuation	observations → `evacuation_dynamique.jsonl`	suivre l'incendie pas à pas et recommander, à chaque instant, une route sûre ou la mise à l'abri	rejeu SOSA, Dijkstra sur le graphe dynamique
M6 agent	questions, demandes → `answers.jsonl`, `audit_log.jsonl`	répondre en langage naturel et traiter des demandes de modification sans jamais violer une règle de sécurité	LLM local + outils + veto SHACL

Outils / stack technique : Python 3.11, ifcopenshell, shapely, numpy, networkx, rdflib, pySHACL, HiGHS (highspy), plotly, Ollama (qwen2.5:14b), tous libres. 133 tests automatisés (pytest). Reproductibilité vérifiée : deux runs neufs redonnent des sorties identiques (graphes isomorphes) ; le journal du LLM rejoue M6 à l'identique.

2. Incohérences de la maquette

Comment elles sont détectées. La détection est répartie en couches indépendantes, chacune avec sa propre source d'information, si bien qu'une erreur de la maquette apparaît comme un **désaccord** entre elles :

1. **Contrôle des attributs** (`m1_geometrie/anomalies.py`, 7 règles) : noms dupliqués, fonction contredite par les propriétés, booléens écrits en texte, rattachement à un niveau incohérent avec l'altitude, élément rattaché à aucun niveau, dimensions d'escalier hors norme.
2. **Recoupement de trois sources de connectivité** (`connectivite.py`) : limites d'espace de l'IFC, chaîne mur → ouverture → porte de Pang, contact géométrique. Un passage est retenu à la majorité ; un désaccord n'est une anomalie **de la maquette** que si la source IFC est minoritaire (sinon, 2 fausses anomalies sur les cages d'escalier).
3. **Invariants** : tout espace occupable atteint une sortie ; une porte relie deux espaces d'un même niveau.
4. **SHACL** sur le graphe (M2) : 11 formes maison (symétrie des liaisons, unicité des GlobalId, surfaces positives…).
5. **Vérification à la main** dans BIMvision (`docs/verification_bimvision.md`), qui a révélé une anomalie de plus, ajoutée ensuite aux règles.

Résultat : 16 anomalies écrites dans `geometry.json` (élément, description, traitement) et dans le graphe (`ssi:ModelAnomaly`).

Constat	Traitement
Deux espaces nommés `B202` ; « Bedroom 2 » porte le numéro Revit `B203`	identification par GlobalId ; ambiguïté signalée à l'agent (Q08)
B105 « Room » alors que son OmniClass est « Stairway » (jumeau de A105 « Stair »)	traité comme cage d'escalier
`IsFireExit` vaut le texte « IsFireExit » sur les 14 portes	ignoré ; sorties par géométrie + `IsExternal`
Escalier B rattaché au toit, base à 0 m ; marches données en pieds dans un fichier en mètres	replacé sur Level 1 ; géométrie utilisée
Porte A204–A205 aussi reliée au couloir A201 par l'IFC (pas sa symétrique dans B)	écartée par le vote des 3 sources
Porte A202–A201 rattachée à aucun niveau (BIMvision n'en montre que 7 au Level 2)	niveau déduit du placement (trouvé par la vérification manuelle)
Limites de A201 / B201 décalées de 6 à 10 m ; 12 limites « EXTERNAL » entre deux espaces ; cages qui empiètent de 0,15 m² sur une chambre	contact géométrique ; limites traitées comme internes ; signalé
Surface géométrique 7 à 23 % sous « GSA BIM Area »	convention de mesure, pas une anomalie ; Q05/Q10 sur « GSA BIM Area »
Risque : 7 espaces seulement ont un comptage ; 6 espaces dangereux à 600 s n'en ont pas	signalé à chaque pas (`espaces_dangereux_sans_comptage`)

3. Écarts à l'article IWCMC 2024

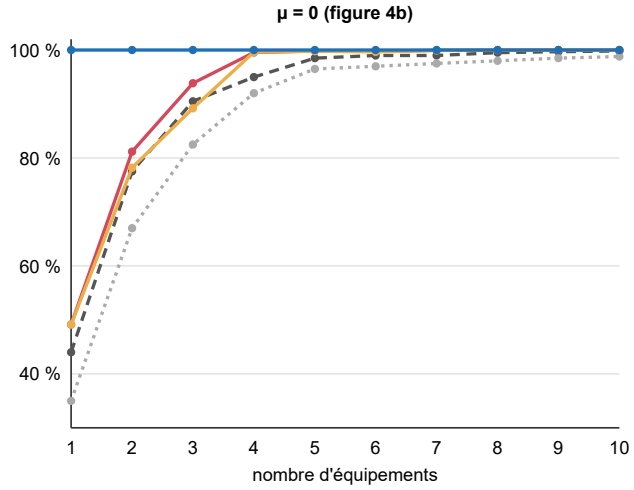
Modèle reproduit. - *Environnement* : grille de 16 × 34 × 24 voxels de 0,5 × 0,5 × 0,25 m (le duplex sur deux niveaux) ; chaque voxel obstacle porte le matériau de l'élément IFC qui l'occupe ; 932 positions de pose = impacts de rayons verticaux sur les plafonds, au-dessus des espaces occupables. - *Propagation* (éq. 2, WiFi 2,4 GHz) : $PL[o, p] = PL_o + 20 \log\|o - p\| + \Sigma L_{obs} + X_\sigma$, avec PL_o = 37,15 dB, X_σ gaussien (σ = 3,26 dB), et une perte par obstacle traversé selon le matériau (tableau I) : plaque de plâtre 0,32 dB, porte en bois 0,23, mur porteur 0,22, mur béton 0,18. - *Score* : un voxel est couvert si la puissance reçue dépasse S_{RX} = −65 dBm (E_{TX} = 20 dBm) ; $F_{obj} = \Sigma_p (\alpha v''_p + \beta \cdot \mathbb{1}[v''_p > 0])$; problème : min |O|, max F_{obj}, sous μ-connexité du réseau $\Gamma(G_O) \geq \mu$. - *Algorithme* : PREP-NSGA, paramètres du tableau II (300 itérations, population 40, 160 mutants, γ = 2 m, graine 0).

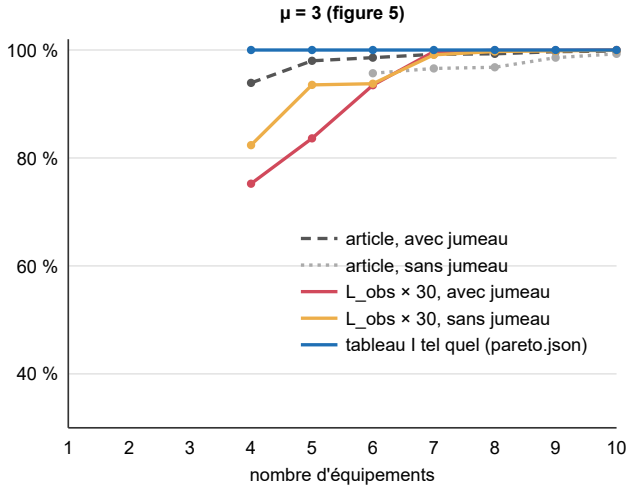
À quoi il est comparé. Aux figures 4 (μ = 0) et 5 (μ = 3) de l'article : couverture et qualité de lien en fonction du nombre d'équipements, pour les **4 scénarios** imposés : avec jumeau (matériaux réels) ou sans jumeau (matériau le plus absorbant partout, solution évaluée ensuite dans le jumeau), × μ = 0 ou μ = 3.

Résultat 1 – avec le tableau I tel quel, le problème est trivial. Un seul équipement couvre 100 % du duplex dans les 4 scénarios (Q20 = 1, `pareto.json`) : la perte maximale est d'environ 62,5 dB en espace libre sur 19 m, plus au plus 10 dB d'obstacles, le signal reste au-dessus de −65 dBm partout.

Résultat 2 – campagne de 12 expériences (`comparaison.md`, `python -m twinhyp campagne`) pour chercher ce qui reproduit les figures :

Scénario testé	Conclusion
L_{obs} en dB par mètre traversé (au lieu de par voxel)	toujours 100 % dès 1 équipement : n'explique pas l'écart
S_{RX} de −65 à −45 dBm ; domaine d'un seul niveau ; tirage du *shadowing* ; pas du lancer de rayons (5 → 1,25 cm : 91 % de rayons identiques) ; autres technologies du tableau I	aucun n'explique l'écart
L_{obs} × 30 à 40 (≈ 10 dB par voxel de plâtre, valeur réaliste d'un mur)	**figure 4 reproduite** : 49 % pour un équipement (article 44 %), écart moyen 0,02 à 0,035, Q20 = 3 comme l'article
Part du matériau le plus absorbant ramenée de 47 % à 21 % (article : 18 %)	le gain du jumeau triple à 2 équipements (+5 points)
PREP-NSGA à μ = 3, 100 / 300 / 1 000 itérations, 5 graines	même optimum local ; dispersion de 7 à 16 points entre graines





μ = 0 (figure 4) : avec × 30, la forme de la courbe est retrouvée et le jumeau améliore la couverture dans le bon sens (2 équipements : 81 % contre 78 % ; 3 : 94 % contre 89 %), mais 2 à 3 fois moins que dans l'article (44 % contre 35 % à 1 équipement ; chez nous, la même position est choisie avec ou sans jumeau).

μ = 3 (figure 5) : l'article trouve 94 % dès 4 équipements avec le jumeau et **aucune solution** sans jumeau à 4 et 5 équipements ; nous ne retrouvons ni l'un ni l'autre.

Couverture à μ = 3	**4 équipements**	**5**	**6**
Article, avec / sans jumeau	94 % / aucune	98 % / aucune	99 % / 96 %
Tableau I tel quel, avec / sans	100 % / 100 %	100 % / 100 %	100 % / 100 %
L_{obs} × 30, avec / sans	75 % / 82 %	84 % / 94 %	94 % / 94 %

Hypothèses sur les causes des écarts, de la plus à la moins probable. 1. **L'ordre de grandeur de L_{obs}** : l'implémentation d'origine applique sans doute la perte à chaque *échantillon* du lancer de rayons (pas de quelques cm), d'où un facteur 20 à 40 ; c'est la seule hypothèse qui reproduit la figure 4. 2. **La part de plâtre** : 47 % de nos voxels obstacles contre 18 % dans l'article ; plus cette part est grande, plus « sans jumeau » ressemble à « avec jumeau » (l'article le prédit lui-même), d'où un gain du jumeau plus faible. 3. **L'algorithme sous μ = 3** : sans croisement et avec peine de mort, peu de mutants sont faisables ; PREP-NSGA se bloque dans des optimums locaux, au point que « sans jumeau » fait mieux que « avec » à 4–5 équipements. La référence [7] (PREP-NSGA) est absente du kit. 4. **La formule de la qualité moyenne de lien**, illisible dans le PDF, et le domaine « 16 × 34 × 10 » (un ou deux niveaux ?).

Ces 4 points sont posés comme questions aux auteurs dans `iwcmc.md` (10 ambiguïtés de lecture, 14 décalages chiffrés). `pareto.json` contient la reproduction avec le tableau I tel quel ; la variante × 30 est dans `sorties/iwcmc/pareto_facteur30.json`.

4. Ontologie : comparaison avec celle de Pang

Aspect	**Ontologie FE de Pang**	**Notre modèle**
Espace de noms	ad hoc, non aligné	socle `ssi:` du kit, étendu par subsomption (`ssi:Door` ⊑ `s4bldg:Door`, `ssi:Dwelling` ⊑ `bot:Zone`) ; jamais d'équivalence, qui engage dans les deux sens
Topologie	`hasStorey` / `hasSpace` propres	BOT ; `isConnectedTo` matérialisé dans les deux sens (SPARQL sans raisonneur)
Distances	valeur sur la relation	**`ssi:EvacuationDistance` réifiée** (origine, destination, élément franchi, distance, valeur de Pang à côté)
Observations	« inspirées de SOSA »	SOSA natif : `SD_A103 sosa:madeObservation obs . obs a sosa:Observation ; sosa:observedProperty ssi:SmokeObscuration ; sosa:hasFeatureOfInterest A103 ; sosa:hasSimpleResult 0.0328 ; sosa:resultTime "…"^^xsd:dateTime`
Danger	statut + distance portée à 1 000 m	statut + **contrainte dure F8**
Validation	aucune	SHACL F1–F8 et 11 formes maison, pour anticiper les formes cachées
Équipements	capteurs génériques	détecteurs (fumée / chaleur), déclencheurs, caméras, zones de détection, relais

Choix défendu : la distance réifiée. Une distance d'évacuation dépend de trois choses (départ, arrivée, passage franchi) et porte plusieurs valeurs (la nôtre, celle de Pang) : une propriété binaire ne peut pas le dire. La réification permet à F7 de contrôler chaque distance, et à la stratégie 2 de construire le graphe pondéré par une seule requête.

Avantages de notre modèle : interopérable (BOT, SOSA, SAREF : un autre outil peut lire le graphe) ; vérifiable (un graphe incohérent est rejeté avant tout calcul) ; le danger n'altère pas la géométrie (aucune distance falsifiée) ; les équipements de sécurité et leurs règles (EN 54 simplifiée) sont représentés. **Inconvénients** : un graphe plus lourd (30 000 triplets avec les observations, contre quelques centaines pour la topologie) et plus lent à valider (8 s par validation SHACL sur le duplex, 42 s sur un bâtiment de 99 espaces), des requêtes plus longues à écrire ; le modèle de Pang, plus simple, se met à jour plus vite et donne toujours une route (au prix d'une route parfois dangereuse).

5. Déploiement SSI (M4)

Pourquoi des petits MILP successifs plutôt qu'un MILP global. Les sous-problèmes sont presque indépendants : la détection, les déclencheurs et la vidéo ont chacun leurs contraintes, et seul le réseau les relie. Un MILP global couplerait plus de 8 000 variables binaires et la connectivité du réseau (non linéaire). Décomposé, chaque problème est petit (de 4 à 7 184 variables), **résolu à l'optimum prouvé** en 0,1 à 13 s, avec un statut et un gap que le schéma `devices.json` demande, et chaque formulation tient au tableau. Le prix : l'optimum global n'est pas garanti. HiGHS est le solveur libre (MIT) qui résout chacun de ces MILP.

- **Détecteurs** (*set cover*) : 932 positions, 944 cellules de 0,5 m des espaces occupables, N(i) = positions du même espace à moins de r = 5 m en vue directe (la fumée ne traverse pas les cloisons) : min $\Sigma_j x_j$ sous $\Sigma_{j \in N(i)} x_j \geq 1$ pour toute cellule i, $x_j \in \{0, 1\}$ → **20 détecteurs**, un par espace occupable (F3).
- **Déclencheurs manuels** : distance **de marche** ≤ 10 m (graphe de visibilité), un par sortie (usage NF S 61-970 ; le minimum serait 2 pour 4 sorties) → **4**, à 0,4 m de leur sortie (F4).
- **Caméras**, modèle du PIMRC transposé en 3D : 7 184 candidats (positions × 8 azimuts × 2 inclinaisons), cône de 90° × 60°, portée 8 m, ligne de vue dans les voxels ; 182 cellules sur les 20 chemins d'évacuation. Phase 1 : min Σz_c sous $\Sigma z_c \geq 1$ sur les caméras c qui voient k, pour toute cellule k (couverture 100 %) → N* = **18**. Phase 2 : à budget N*, **max-min** du *full-view* par chemin (cellule vue de face, à ±45° de la direction de marche), puis max du total.
- **Zones** : une par niveau et par logement (4, F6). **Réseau** : modèle radio de M3 entre équipements → connexe (λ = 41), aucun relais.

Résultat : 42 équipements, couverture vidéo de 100 % sur chaque chemin, *full-view* global de 69 %. Le minimum par chemin est de 0 % : le MILP **prouve** qu'aucune configuration de 18 caméras ne fait mieux (B102 et B103). Compromis documenté : +1 caméra → 10 %, +2 → 62,5 %.

6. Évacuation dynamique : pénalité ou contrainte dure (M5)

Le scénario. 61 pas de 10 s, feu dans la cuisine A103, rejoués selon l'algorithme 1 de Pang : à chaque pas, les observations sont insérées en SOSA et les états mis à jour par SPARQL UPDATE (alarme : obscurcissement > 0,0328 /m ; `Dangerous` : T > 60 °C ou CO > 1 000 ppm). Alarme à **70 s** ; A101 devient dangereux à **180 s**, et avec lui les deux sorties du logement A.

Les deux stratégies.

	Stratégie 1 : sortie la plus proche	**Stratégie 2 : Dijkstra sur le graphe dynamique**
Principe	route calculée une fois, sans tenir compte du feu	recalculée à chaque pas, espaces `Dangerous` exclus
Logement B	évacue (B102, B103 vides dès 150 s)	identique
A202 (2 pers.) et A203 (1 pers.), de 180 à 600 s	route A202 → A201 → A105 → **A101 dangereux** : 86 lignes, 129 personnes × pas exposées	**aucune route sûre** sur ces 86 lignes

Les deux traitements du danger quand il n'y a plus de route sûre (A202 à 180 s).

	Pénalité de Pang (1 000 m)	**Contrainte dure (F8)**
Route	A202 → A201 → A105 → A101, **1 026 m, à travers le feu**	`route = null`
Consigne	« suivre la route »	mise à l'abri, porte fermée, position signalée aux secours
Ce que voit l'opérateur	une distance anormale, facile à prendre pour une route valide	l'absence de route, explicitement

Un « détour réactif » (suivre le plan et s'arrêter avant le premier espace dangereux) ferait pire : l'occupant serait bloqué dans la cage A105, elle-même dangereuse à 250 s.

Conclusion. Les deux approches coïncident tant qu'une route sûre existe ; elles divergent précisément dans le cas critique. La pénalité masque l'absence de route sûre derrière un grand nombre et propose de traverser le feu ; la contrainte dure dit la vérité (« aucune route sûre ») et déclenche la bonne consigne. La route « pénalité » reste jointe à chaque ligne pour comparaison. Limite commune : Pang raisonne en valeurs instantanées ; une évaluation en dose cumulée (ISO 13571, ASET/RSET) serait plus juste. Réponses : Q15 = 70 s, Q16 = A101, A102, A103, A105, A201, Q17 = A202 → A201 → A105 → A101, Q18 = 180 s, Q19 = A202, A203.

7. Agent neuro-symbolique (M6)

Structure. Un LLM open-weight local (qwen2.5:14b, 4 bits, Ollama, température 0) dans une boucle écrite à la main (pas de framework), et des outils qui font tout le calcul.

Outil imposé	Notre réalisation	Appelé par
Requête SPARQL	`sparql` (lecture seule) et 18 outils paramétrés sur des gabarits SPARQL (espaces d'un niveau, portes extérieures, surfaces, itinéraires…)	le LLM
Planificateur d'évacuation	`route_evacuation`, `itineraire`, `instant_sans_route_sure` (Dijkstra de M5)	le LLM
Optimiseur	MILP de M4 (replacer des détecteurs, D3) ; front de Pareto de M3	le moteur de demandes
Validation SHACL	validateur appliqué à une copie du graphe pour chaque candidat	le moteur de demandes

L'optimiseur et SHACL ne sont pas laissés au LLM : il ne peut donc pas les contourner. Chaque appel au modèle (prompt complet, réponse, durées, jetons, digest du modèle) est journalisé dans `llm_journal.jsonl`.

Pipeline des questions. (1) Le LLM reçoit la question et un résumé du bâtiment généré par SPARQL ; (2) il choisit un outil et ses arguments ; (3) l'outil calcule, **sa sortie est la réponse** (le LLM ne recopie jamais un identifiant) et les requêtes exécutées sont tracées ; (4) si l'outil renvoie une erreur ou un mauvais type, l'erreur est rendue au LLM (3 tours au plus).

Pipeline des demandes. (1) Le LLM traduit la demande en un plan JSON contraint (action, espace, niveau, instant) ; (2) des règles calculent les triplets à ajouter et à retirer ; (3) SHACL valide une **copie** du graphe ; (4) si une forme est violée, le moteur génère des adaptations (relais, détecteurs temporaires, route sûre) ; (5) les candidats sont classés par ordre lexicographique : sécurité (conformité, couverture de détection) > accessibilité (F4, F8) > sobriété (nombre d'équipements) > satisfaction de la demande ; (6) la modification retenue est appliquée et inscrite dans `audit_log.jsonl` ; (7) le LLM rédige la justification.

Résultats. 20 / 20 réponses identiques à une référence calculée sans LLM ; sur Q08, l'outil signale que « B202 » désigne deux espaces et le modèle se corrige seul (« Bedroom 2 »).

Demande	Décision	Raison
D1 cuisine en détecteurs de chaleur	acceptée	`HeatDetector` ⊑ `Detector` : F3 reste vraie
D2 déplacer l'équipement du couloir A201	refusée (veto)	F3 : A201 ne serait plus détecté ; un relais est inutile, le réseau est déjà connexe
D3 retirer les détecteurs du Level 2	acceptée avec adaptation	F3 violée ; 10 détecteurs de chaleur temporaires placés par le MILP de M4 (insensibles aux poussières)
D4 route habituelle à 200 s (5 espaces)	refusée, mise à l'abri	F8 : la route traverse A101 ; à 200 s les deux sorties du logement A sont dangereuses

Fumée ou chaleur (`fumee_chaleur.md`) : la chaîne rejouée avec des détecteurs de chaleur donne une alarme à 80 s (classe A1R) ou 130 s (A1) contre 70 s. D1 a donc un coût que SHACL ne voit pas ; la fumée reste le choix par défaut.

8. Généricité (second IFC)

La chaîne a tourné sur le bâtiment de bureaux de la même série NIBS (`Office_A_20110811.ifc` : 99 espaces, 102 portes), avec un scénario synthétique (`python -m twinhyp scenario`) et les questions du kit reformulées (`generalite.md`). Ce second IFC a révélé 7 cas absents du duplex, corrigés par des **règles générales** (le duplex est inchangé) : gaine d'ascenseur sans aucune liaison, portes de cabines de toilettes prises pour des sorties, local technique sans plafond, cages d'escalier découpées par niveau, « RR » pour *restroom*, référence de M6 facultative, validation SHACL de D4 en une passe. Résultat : 149 détecteurs, 6 déclencheurs, 75 caméras (100 % des chemins), tout conforme.

9. Matériel et temps de calcul

Portable AMD Ryzen 7 7730U (8 cœurs), 15,4 Go de RAM, sans GPU (inférence sur CPU) ; Windows 11, Python 3.11 (conda), Ollama 0.40, qwen2.5:14b en Q4_K_M (9 Go). Chaque run écrit `execution.json` (versions, machine, durée de chaque étape).

	Duplex	Bureaux NIBS (mode rapide)
M1	10 s	29 s
M2	0,2 s (+ 4 s de validation)	1 s
M3	176 s (dont la variante × 30)	128 s (40 positions)
M4	19 s (+ 4 s)	165 s
M5	20 s (+ 21 s)	93 s (+ 130 s de validation finale)
M1 → M5	**4 min 20**	**≈ 9 min 15**
M6 (LLM)	1 192 s (questions 607 s, demandes 568 s, 8 validations SHACL)	non mesuré (mémoire)

Banc d'essai des modèles locaux : qwen2.5:14b 20 / 20 ; llama3.1:8b 2 / 4 sur un sous-ensemble (mauvais outil, booléen envoyé en texte) ; mistral et qwen3:4b écartés à l'essai initial.

10. Limites de l'agent et pistes vers un produit

En situation de crise, un opérateur attend une réponse en **quelques secondes**, fiable, même si une partie du système tombe. L'agent actuel en est loin sur plusieurs points.

Limite actuelle	Source	Amélioration
Trop lent : 15 à 30 s par question, 240 s au premier appel (chargement), ~10 min pour les 4 demandes	LLM de 14 B sur CPU ; validation SHACL du graphe entier (8 s par candidat, 42 s sur 99 espaces)	GPU ou modèle plus petit spécialisé ; modèle gardé en mémoire ; validation incrémentale (seules les formes et les nœuds touchés) ; plans d'urgence précalculés pour chaque espace et chaque état
Couverture limitée : 4 types d'actions (changer un type, déplacer, retirer, diriger) ; une question hors des 19 outils passe par le SPARQL libre, moins fiable	outils écrits pour les questions du kit	bibliothèque d'outils plus large ; génération de SPARQL contrôlée par le schéma ; refus explicite quand l'agent ne sait pas
Choix d'outil fragile sur les petits modèles (llama3.1 : 2 / 4, mauvais outil, booléen en texte)	compréhension du LLM	jeu d'évaluation de questions ; ajustement fin ; arguments vérifiés par schéma (déjà en partie)
Justifications approximatives (politique lue à l'envers avant correction, « adaptation » annoncée sans adaptation)	texte libre du LLM	justification générée à partir des faits du moteur (gabarit), le LLM seulement pour la reformulation
Vision instantanée et partielle : un état par pas de 10 s, 7 espaces seulement avec un comptage de personnes, aucune prévision de propagation	données du scénario ; modèle de Pang	flux de capteurs en continu ; prévision du danger (dose cumulée, ASET/RSET) ; signaler les espaces dangereux non observés (déjà fait)
Dépendance au LLM : sans Ollama, plus de réponses en langage naturel	architecture	repli déterministe : les outils et le moteur de décisions fonctionnent sans LLM ; le rejeu du journal existe déjà

Ce qui est déjà au niveau d'un produit : aucune décision ne dépend du texte du LLM (veto symbolique), chaque appel est tracé et rejouable, et la chaîne tourne sur un bâtiment inconnu. Les doutes restants sont dans `doutes.md`, les choix et options écartées dans `choix.md`, le journal daté (impasses comprises) dans `docs/carnet_de_bord.md`.